

Measurement insights

Generated 2026-09-11 18:44 UTC from insights/20260911T184408Z_thesis-extended.

- Profile: **thesis-extended**
- Charts drawn: **31** of 31
- Runs considered: **8**
- Engines: cockroachdb, postgresql

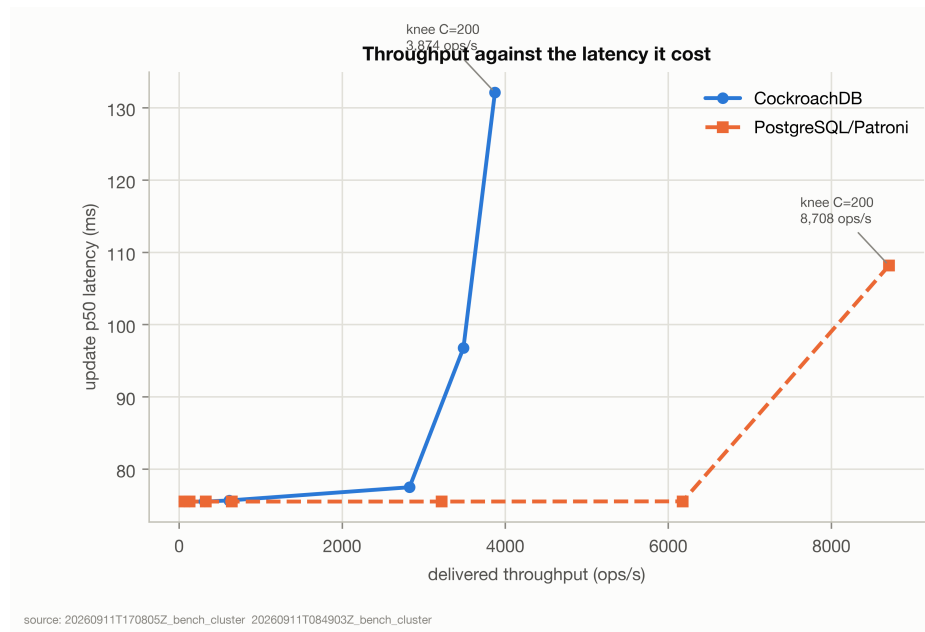
Every chart resolves its inputs through the gated run loader, so a run that failed pre-flight or validation cannot appear in any figure here.

Runs behind this report

run id	engine	profile	phase
20260911T084755Z_p1-network	postgresql	thesis-extended	network
20260911T084903Z_bench_cluster	postgresql	thesis-extended	bench
20260911T092014Z_p4-chaos-recover	postgresql	thesis-extended	chaos-recover
20260911T093806Z_p4-chaos-dead	postgresql	thesis-extended	chaos-dead
20260911T170557Z_p1-network	cockroachdb	thesis-extended	network
20260911T170805Z_bench_cluster	cockroachdb	thesis-extended	bench
20260911T174234Z_p4-chaos-recover	cockroachdb	thesis-extended	chaos-recover
20260911T182217Z_p4-chaos-dead	cockroachdb	thesis-extended	chaos-dead

A - Benchmark and saturation

A1. Throughput-latency curve

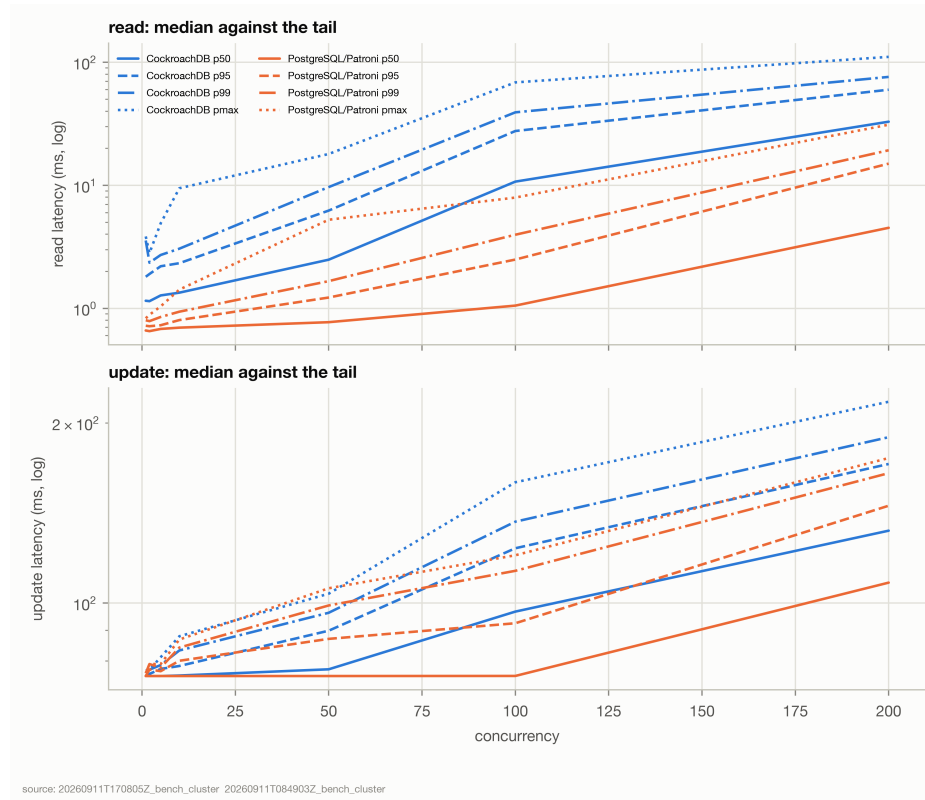


Throughput-latency curve

Each point is one concurrency tier. The curve bends upward at the knee, past which offered load buys queueing rather than throughput. Points are ordered by concurrency, not by throughput, because past saturation the curve genuinely bends backwards.

metric	value
cockroachdb_knee_concurrency	200
cockroachdb_knee_tps	3,874
cockroachdb_peak_tps	3,874
postgresql_knee_concurrency	200
postgresql_knee_tps	8,708
postgresql_peak_tps	8,708

A2. Latency percentile fan

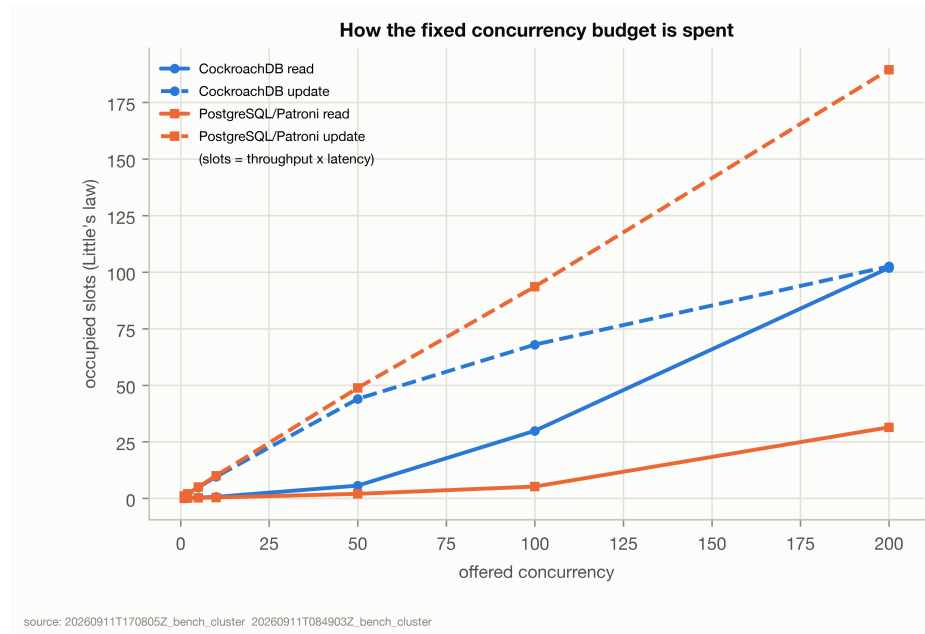


Latency percentile fan

p50, p95, p99 and max for each operation type, per engine, on a log axis. A fan that opens with concurrency is queueing; one that stays parallel is a shifted floor. Operation types are never pooled.

metric	value
cockroachdb_read_p99_max_ms	76.05
postgresql_read_p99_max_ms	19.27
cockroachdb_update_p99_max_ms	189.3
postgresql_update_p99_max_ms	164.9

A3. Concurrency slot occupancy

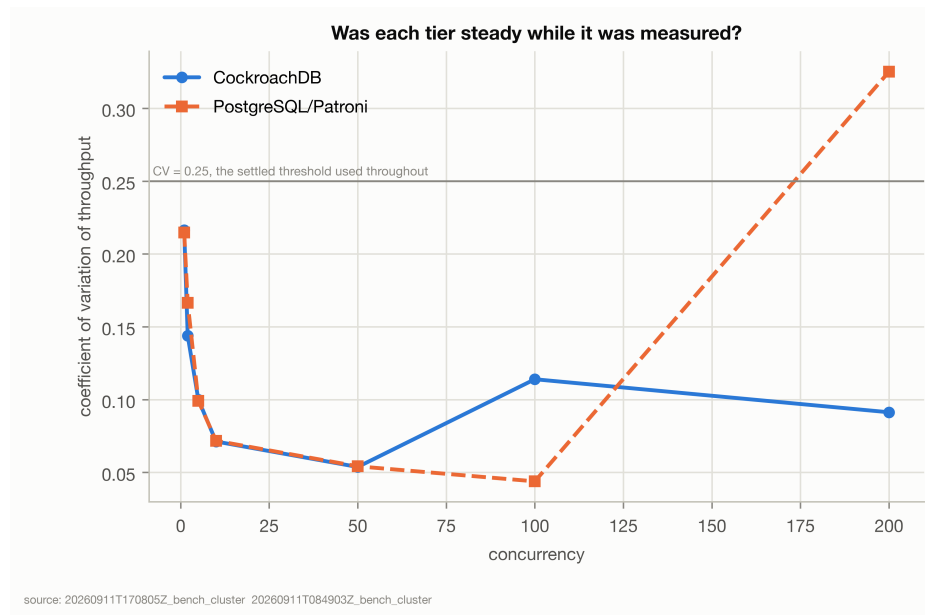


Concurrency slot occupancy

Little's law applied per operation type: an operation occupies throughput x latency of the client's fixed concurrency budget, and both factors are measured per operation rather than assumed from the configured mix. Where reads grow expensive they crowd out writes for the same slots, which is a different statement from either one simply being slow.

metric	value
cockroachdb_read_slots_at_max_c	101.9
cockroachdb_update_slots_at_max_c	102.6
postgresql_read_slots_at_max_c	31.4
postgresql_update_slots_at_max_c	189.5

A4. Steady-state stability

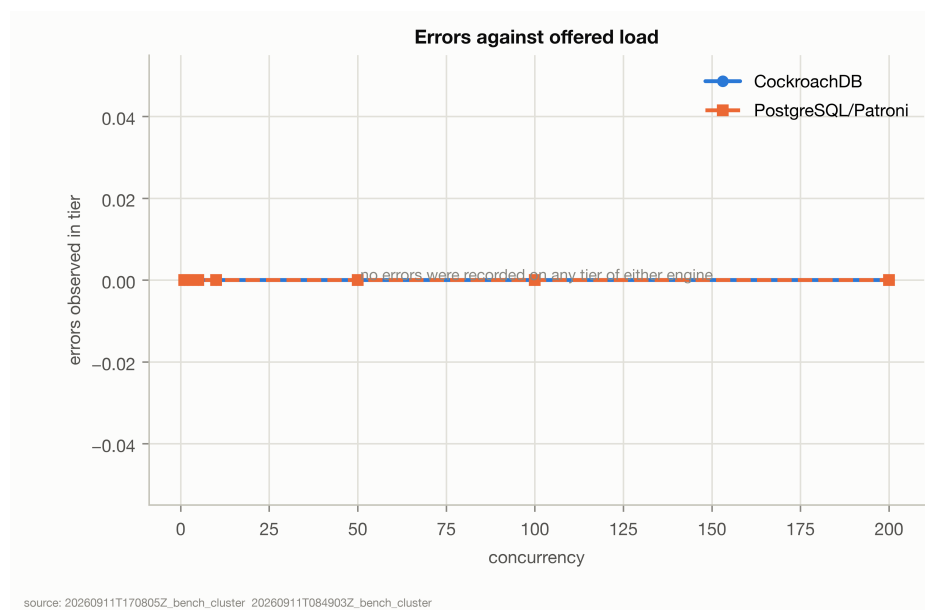


Steady-state stability

Within-tier coefficient of variation of throughput. A tier above the line was still moving while it was being recorded, and its mean is a average over a transient rather than a steady state.

metric	value
cockroachdb_max_cv	0.216
cockroachdb_worst_tier	1
postgresql_max_cv	0.325
postgresql_worst_tier	200

A5. Error rate against load

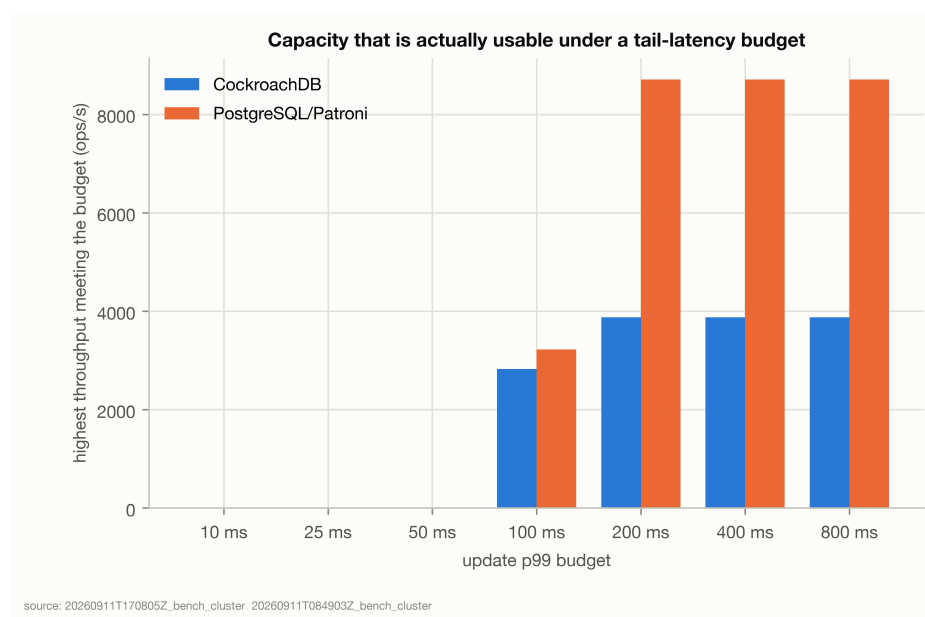


Error rate against load

Errors per tier. The bench sweep runs without `-tolerate-errors`, so a non-zero count here would mean the generator survived something it was not configured to absorb; zero is the expected result and is what makes the throughput figures quotable.

metric	value
cockroachdb_errors_total	0
postgresql_errors_total	0

A6. Throughput under a latency budget



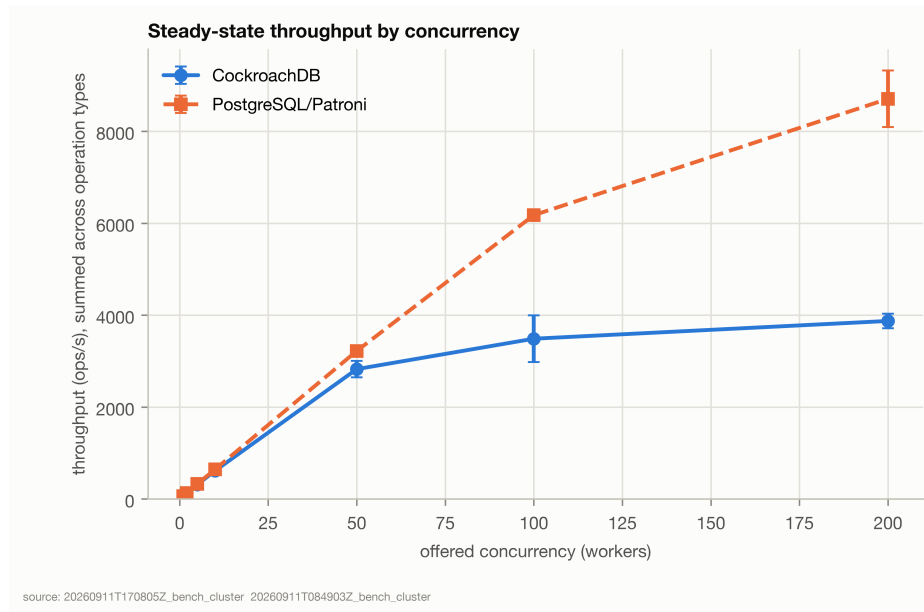
Throughput under a latency budget

The highest measured throughput whose p99 stayed inside each budget. A zero bar means no tier met that budget at all. This restates the same tiers as A1 in the form a service owner buys: peak throughput at an unbounded tail is not deliverable capacity.

metric	value
cockroachdb_tps_under_p99_10ms	0
cockroachdb_tps_under_p99_25ms	0
cockroachdb_tps_under_p99_50ms	0
cockroachdb_tps_under_p99_100ms	2,826
cockroachdb_tps_under_p99_200ms	3,874
cockroachdb_tps_under_p99_400ms	3,874
cockroachdb_tps_under_p99_800ms	3,874
postgresql_tps_under_p99_10ms	0
postgresql_tps_under_p99_25ms	0
postgresql_tps_under_p99_50ms	0
postgresql_tps_under_p99_100ms	3,221

metric	value
postgresql_tps_under_p99_200ms	8,708
postgresql_tps_under_p99_400ms	8,708
postgresql_tps_under_p99_800ms	8,708

A7. Throughput by concurrency



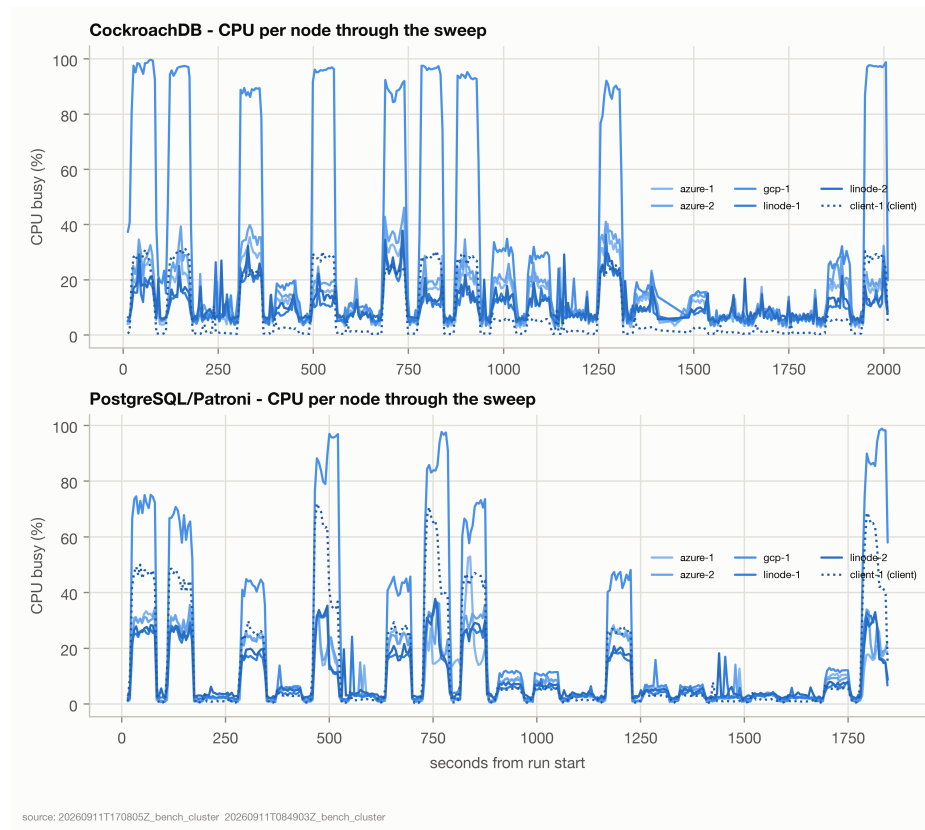
Throughput by concurrency

Mean throughput per tier against the concurrency that produced it. Flattening or falling back past some concurrency is saturation; A1 shows what that saturation costs in latency.

metric	value
cockroachdb_peak_tps	3,874
cockroachdb_peak_concurrency	200
postgresql_peak_tps	8,708
postgresql_peak_concurrency	200

B - Hardware utilisation

B1. Per-node CPU timeline

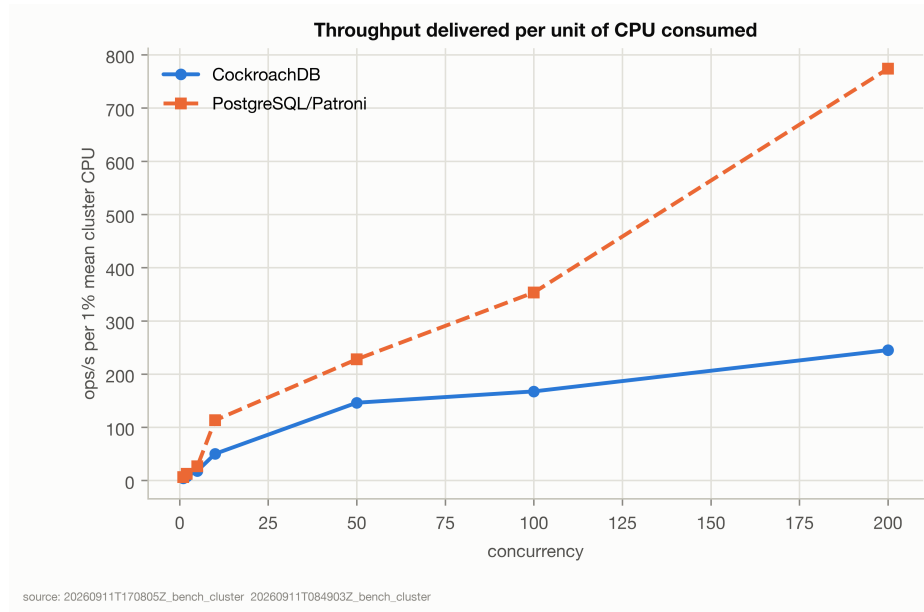


Per-node CPU timeline

CPU busy per node across the benchmark sweep, one panel per engine. The client node is dotted and is not a cluster member; it is shown so a client-side bottleneck can be excluded rather than assumed away.

metric	value
cockroachdb_azure-1_cpu_mean_pct	12.7
cockroachdb_azure-2_cpu_mean_pct	14.8
cockroachdb_gcp-1_cpu_mean_pct	35
cockroachdb_linode-1_cpu_mean_pct	10.3
cockroachdb_linode-2_cpu_mean_pct	11.4
postgresql_azure-1_cpu_mean_pct	9.6
postgresql_azure-2_cpu_mean_pct	9.8
postgresql_gcp-1_cpu_mean_pct	22.8
postgresql_linode-1_cpu_mean_pct	8.8
postgresql_linode-2_cpu_mean_pct	9.6

B2. CPU efficiency

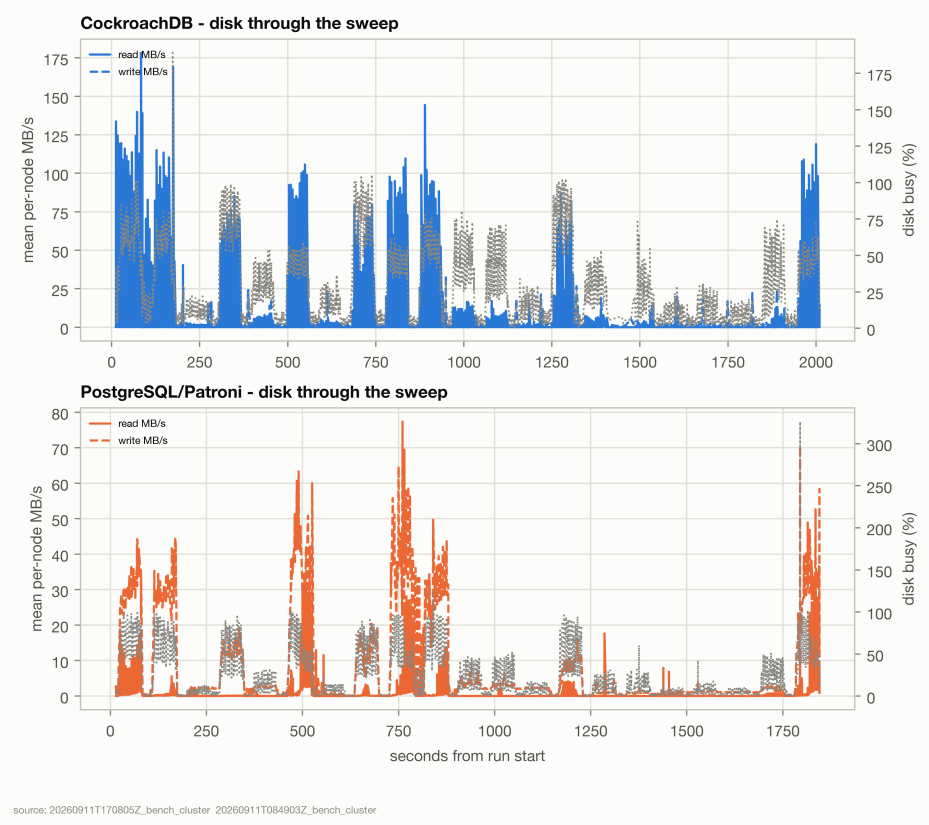


CPU efficiency

Delivered throughput divided by mean CPU busy across the five cluster nodes, per tier. This is the cost-of-replication figure that throughput alone cannot give: an engine can be faster and still be spending more machine to get there. The client node is excluded from the CPU mean.

metric	value
cockroachdb_c1_ops_per_cpu_pct	4
cockroachdb_c2_ops_per_cpu_pct	7.9
cockroachdb_c5_ops_per_cpu_pct	18
cockroachdb_c10_ops_per_cpu_pct	50.1
cockroachdb_c50_ops_per_cpu_pct	146.2
cockroachdb_c100_ops_per_cpu_pct	167.4
cockroachdb_c200_ops_per_cpu_pct	245
postgresql_c1_ops_per_cpu_pct	6.3
postgresql_c2_ops_per_cpu_pct	12.7
postgresql_c5_ops_per_cpu_pct	26.7
postgresql_c10_ops_per_cpu_pct	113.1
postgresql_c50_ops_per_cpu_pct	227.8
postgresql_c100_ops_per_cpu_pct	353.3
postgresql_c200_ops_per_cpu_pct	773.9

B3. Disk I/O and busy time

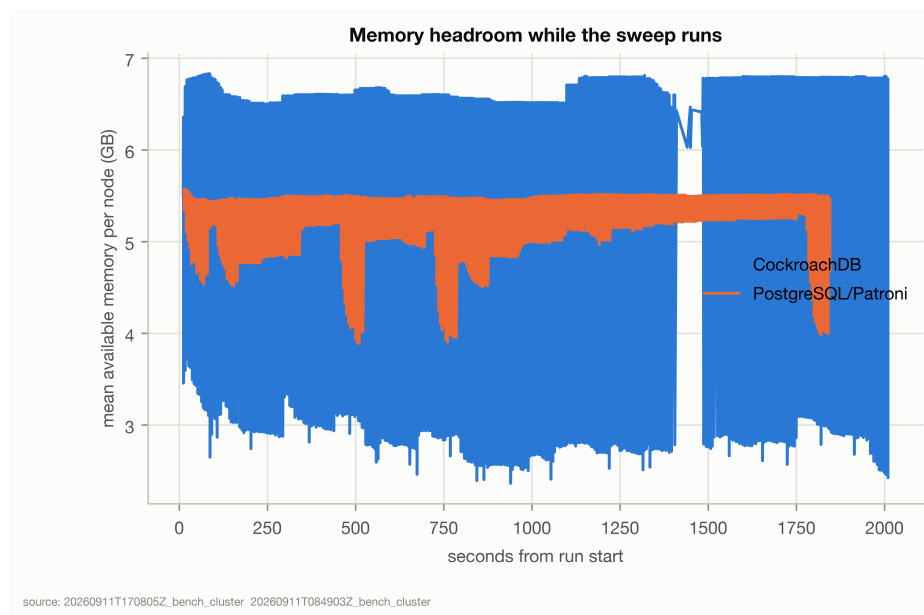


Disk I/O and busy time

Mean per-node disk throughput and busy time across the cluster. At the current profile the working set is roughly 1.5x node RAM, so sustained read traffic here is the evidence that the benchmark is disk-bound rather than served from cache – the assumption the whole profile rests on. A second axis carries busy %; it is dotted and grey because it is a different quantity, not a third series.

metric	value
cockroachdb_disk_busy_mean_pct	30.7
cockroachdb_disk_read_mean_mb_s	5.62
cockroachdb_disk_write_mean_mb_s	3.08
postgresql_disk_busy_mean_pct	25.9
postgresql_disk_read_mean_mb_s	0.71
postgresql_disk_write_mean_mb_s	9.07

B4. Memory headroom

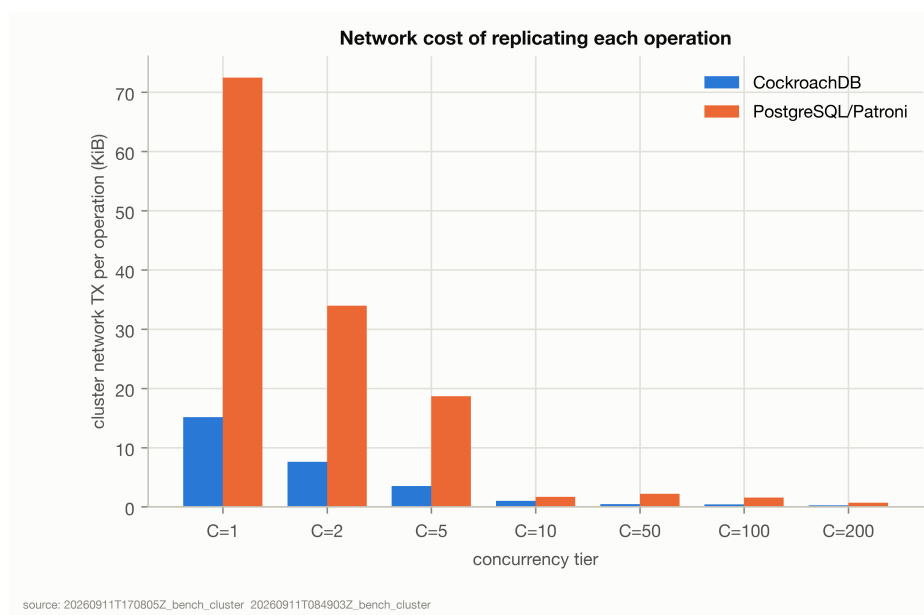


Memory headroom

Mean available memory across the five cluster nodes. Both engines are configured with a cache of a quarter of measured RAM, so this shows the OS page cache absorbing the remainder – and how little headroom is left once a working set larger than RAM is being served.

metric	value
cockroachdb_mem_available_min_gb	2.36
cockroachdb_mem_total_gb	8.32
postgresql_mem_available_min_gb	3.89
postgresql_mem_total_gb	8.32

B5. Replication traffic amplification

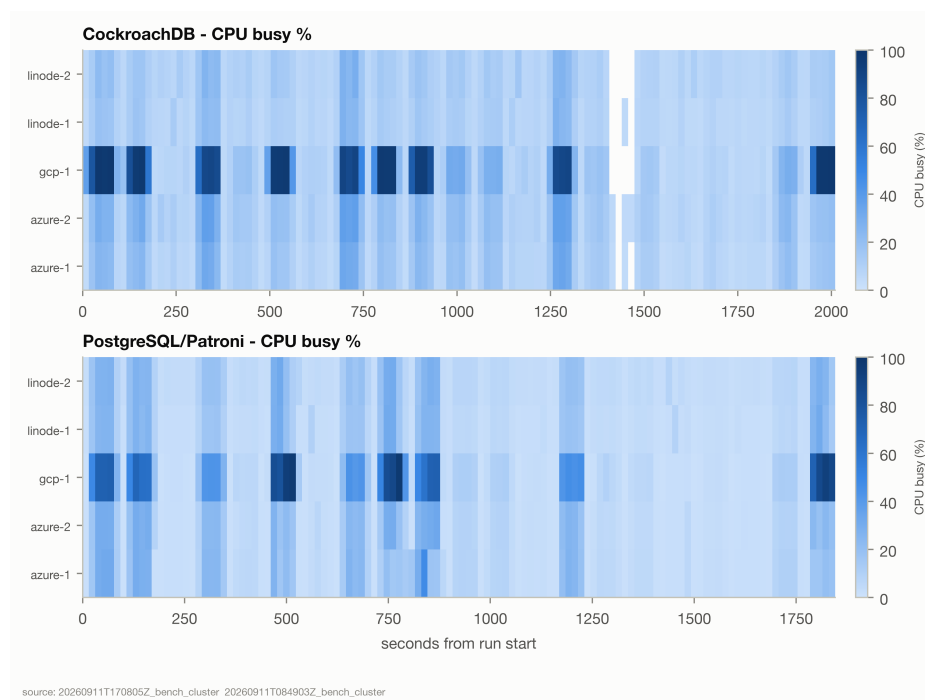


Replication traffic amplification

Total bytes transmitted by the five cluster nodes divided by the operations they served, per tier. This is the closest direct measurement the testbed makes of what each replication design costs on the wire – Raft’s per-range replication against Patroni’s WAL streaming. It is a ratio of two measured rates, so it is insensitive to the tiers having different durations. Read it as an order of magnitude: the link also carries Tailscale and node exporter traffic, which is not separated out.

metric	value
cockroachdb_c1_tx_kib_per_op	15.13
cockroachdb_c2_tx_kib_per_op	7.6
cockroachdb_c5_tx_kib_per_op	3.49
cockroachdb_c10_tx_kib_per_op	1.01
cockroachdb_c50_tx_kib_per_op	0.44
cockroachdb_c100_tx_kib_per_op	0.4
cockroachdb_c200_tx_kib_per_op	0.26
postgresql_c1_tx_kib_per_op	72.46
postgresql_c2_tx_kib_per_op	33.95
postgresql_c5_tx_kib_per_op	18.67
postgresql_c10_tx_kib_per_op	1.7
postgresql_c50_tx_kib_per_op	2.21
postgresql_c100_tx_kib_per_op	1.57
postgresql_c200_tx_kib_per_op	0.68

B6. Cluster utilisation heatmap

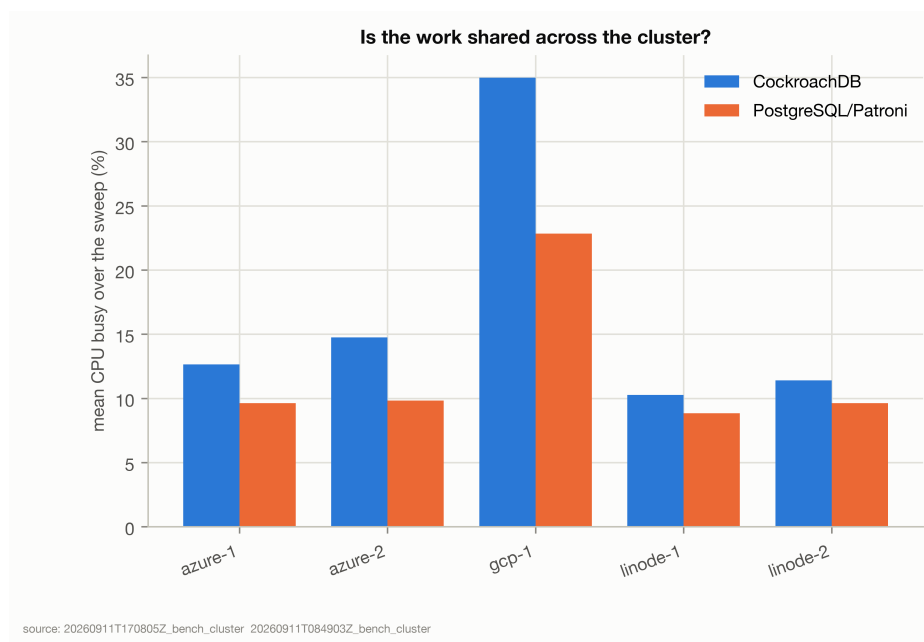


Cluster utilisation heatmap

Every cluster node's CPU on one grid, binned onto a common time axis because the nodes are polled independently. A single bright row is a single busy machine; a uniformly lit panel is work spread across the cluster. One continuous hue, so brightness reads as magnitude rather than as category.

metric	value
cockroachdb_cpu_peak_pct	97.8
postgresql_cpu_peak_pct	97.3

B7. Cluster load imbalance



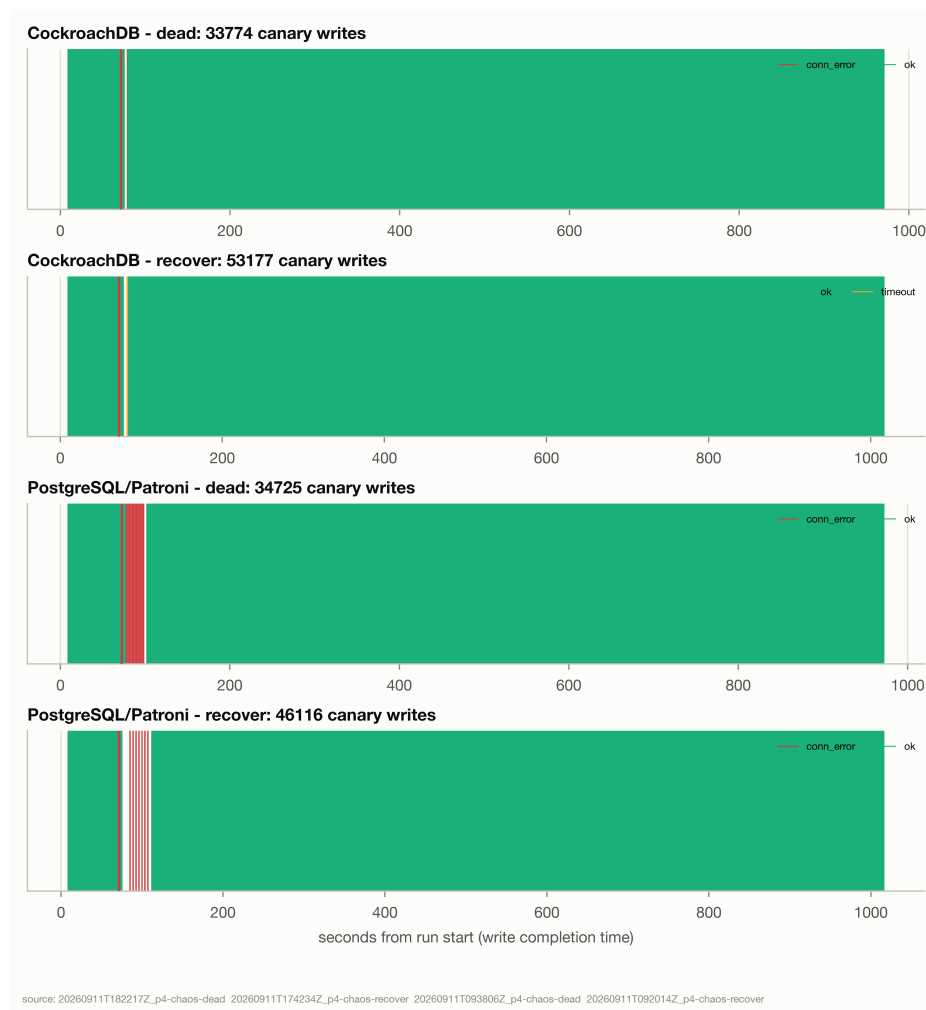
Cluster load imbalance

Mean CPU per cluster node over the whole sweep, with a scale-free coefficient of variation recorded alongside so that an engine which simply runs hotter everywhere is not counted as imbalanced. Read this against the deployment rather than against the engines' reputations: this testbed deliberately pins CockroachDB's leaseholders to the gateway via lease_preferences and pins Patroni's primary to the same node, so neither arm is free to spread work as it otherwise might. What the chart measures is how much of the cluster each engine still uses under that pinning, which is a property of this configuration and must be quoted as one.

metric	value
cockroachdb_cpu_imbalance_cv	0.548
cockroachdb_busiest_node	gcp-1
postgresql_cpu_imbalance_cv	0.441
postgresql_busiest_node	gcp-1

C - Resilience

C1. Probe attempt strip

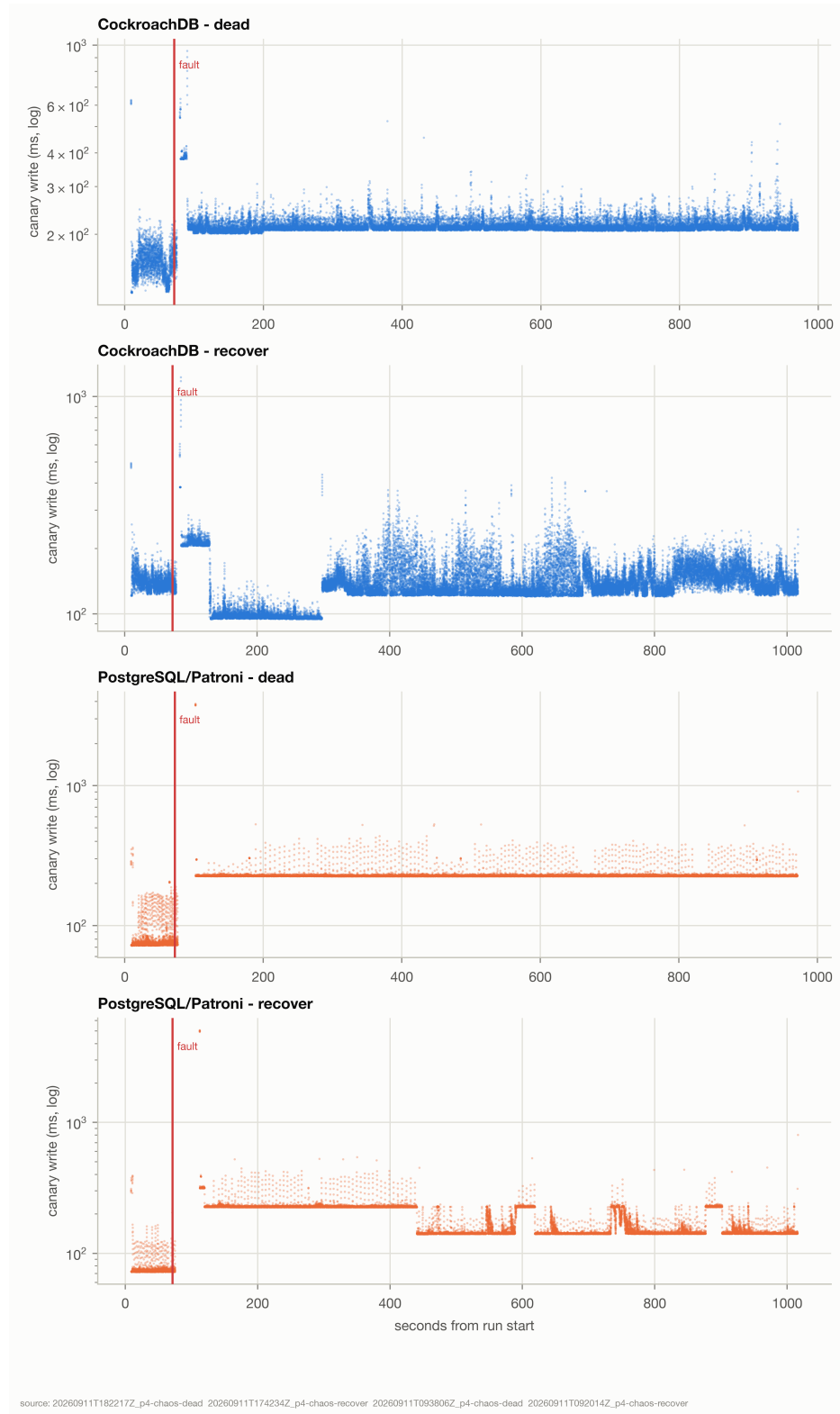


Probe attempt strip

One vertical rule per canary write, placed at the moment it completed and coloured by outcome. The outage is the blank band: this is the outage as directly observed, with no statistic between the reader and the measurement. Writes are placed by completion time, never dispatch time.

metric	value
cockroachdb_dead_attempts	33774
cockroachdb_dead_outcomes	ok=33766, conn_error=8
cockroachdb_recover_attempts	53177
cockroachdb_recover_outcomes	ok=53169, timeout=8
postgresql_dead_attempts	34725
postgresql_dead_outcomes	ok=34613, conn_error=112
postgresql_recover_attempts	46116
postgresql_recover_outcomes	ok=46060, conn_error=56

C2. Probe latency through the fault

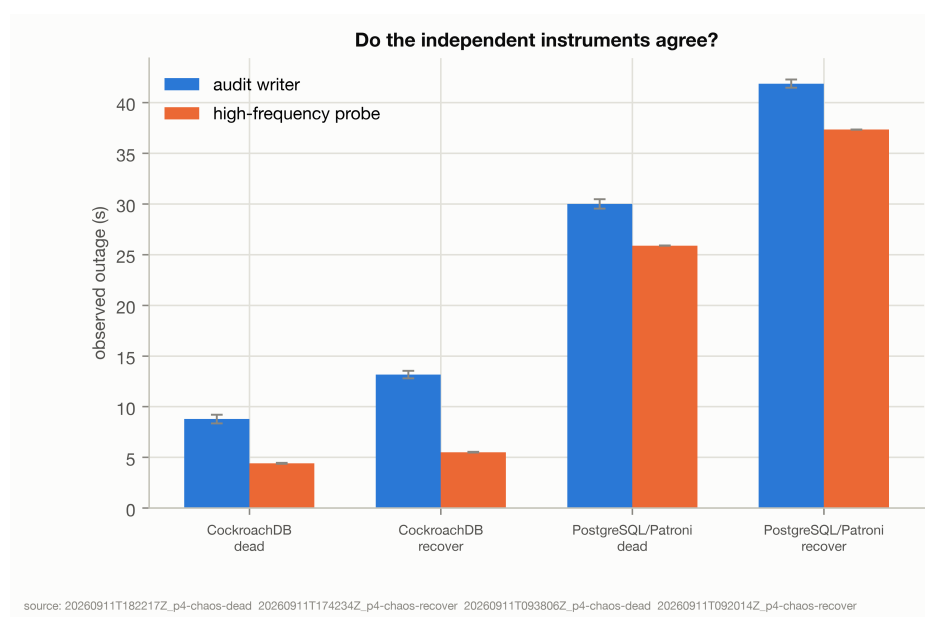


Probe latency through the fault

Duration of every served canary write, log scale. A step in the floor after the fault is a structural change in the write path – the new primary or leaseholder is a different distance away – and is a separate finding from how long writes were unavailable.

metric	value
cockroachdb_dead_pre_fault_p50_ms	154.4
cockroachdb_dead_post_fault_p50_ms	211.2
cockroachdb_recover_pre_fault_p50_ms	139.8
cockroachdb_recover_post_fault_p50_ms	129.4
postgresql_dead_pre_fault_p50_ms	73.53
postgresql_dead_post_fault_p50_ms	226.5
postgresql_recover_pre_fault_p50_ms	73.23
postgresql_recover_post_fault_p50_ms	142.7

C3. Instrument agreement

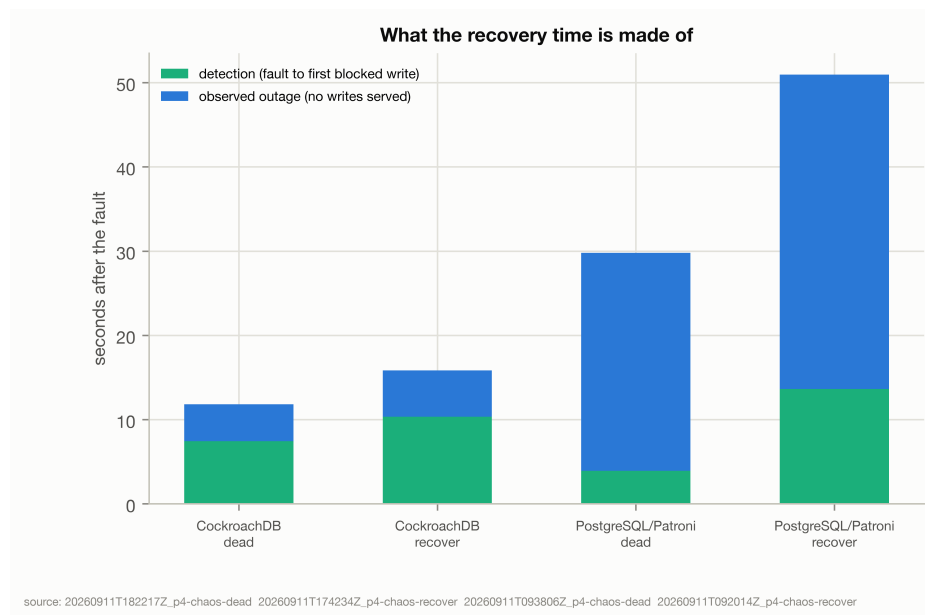


Instrument agreement

The same outage as measured by two instruments that share no code path, with each one's own sampling resolution drawn as its error bar. Agreement within those bars is the defensibility claim: a single instrument can be wrong in the flattering direction and nothing in its own output would show it.

metric	value
CockroachDB_dead	audit_s=8.774, probe_s=4.405
CockroachDB_recover	audit_s=13.16, probe_s=5.507
PostgreSQL/Patroni_dead	audit_s=29.99, probe_s=25.88
PostgreSQL/Patroni_recover	audit_s=41.86, probe_s=37.32

C4. Recovery decomposition

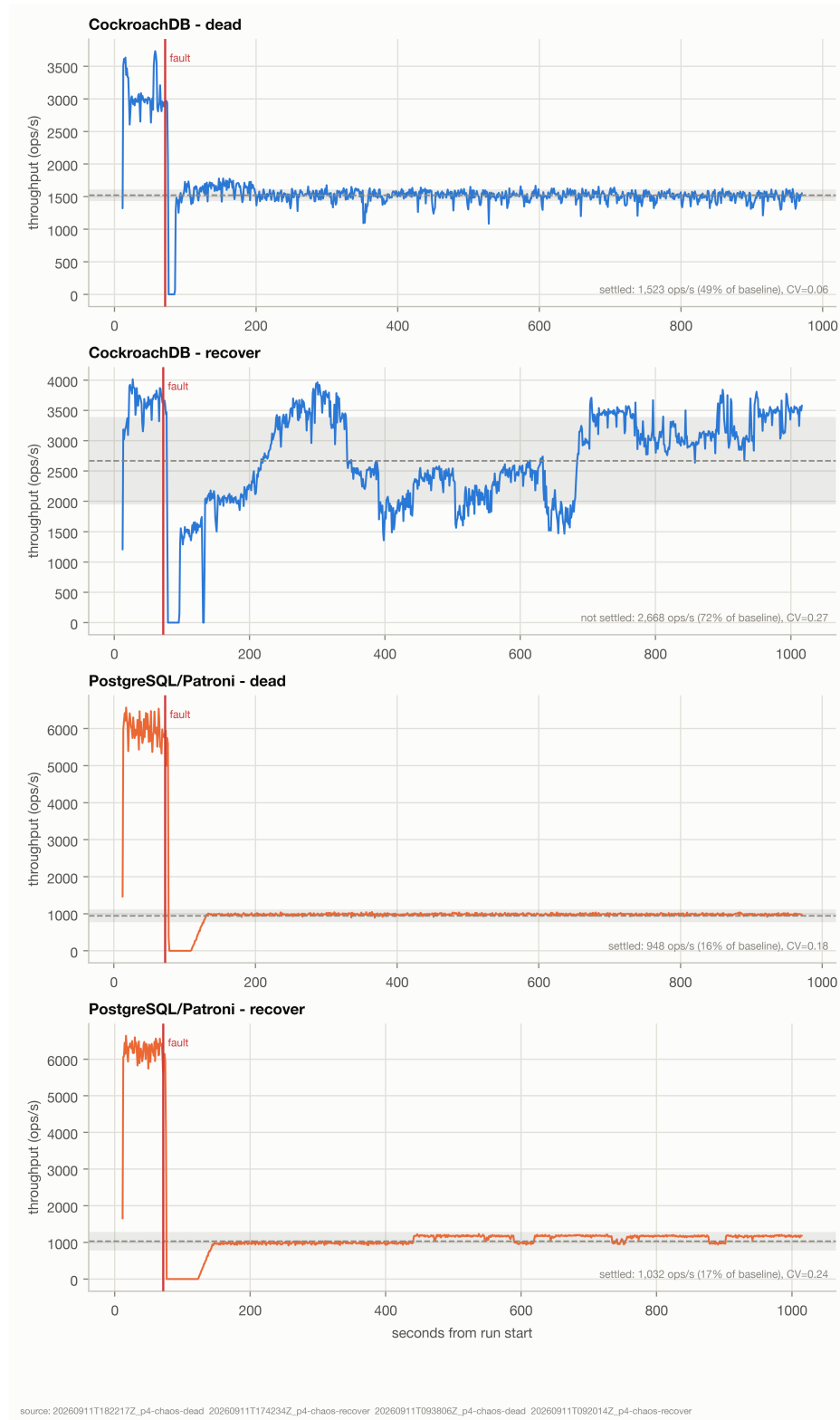


Recovery decomposition

The interval from the fault to the first blocked write is detection, not recovery – the injected command returns before established connections stop working, so writes continue briefly after the fault is nominally in place. Separating the two prevents a fast failover from being credited with a slow detection.

metric	value
CockroachDB_dead	detection_s=7.424, outage_s=4.405
CockroachDB_recover	detection_s=10.33, outage_s=5.507
PostgreSQL/Patroni_dead	detection_s=3.927, outage_s=25.88
PostgreSQL/Patroni_recover	detection_s=13.62, outage_s=37.32

C5. Post-fault settling

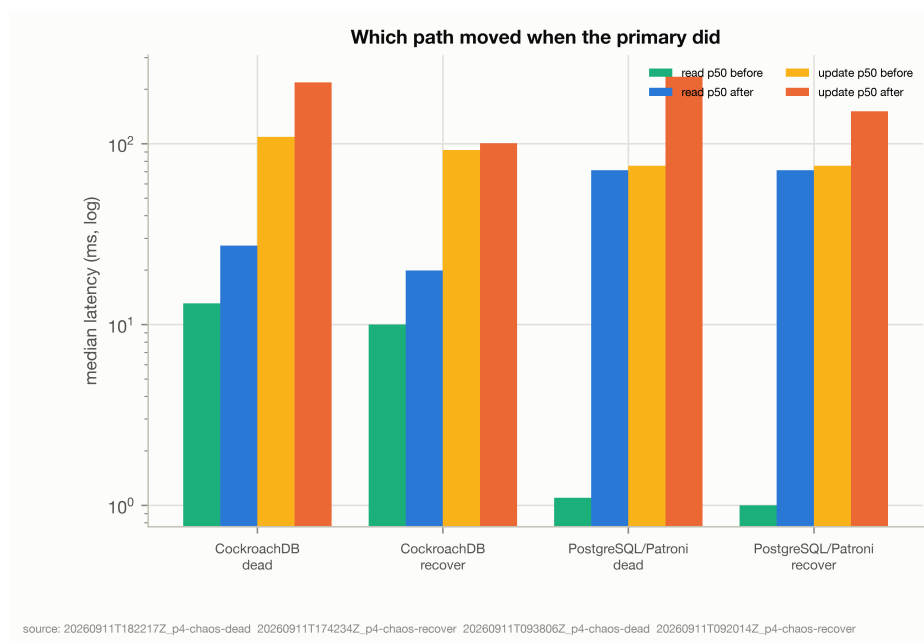


Post-fault settling

Throughput through the fault, with the post-fault mean and its one-sigma band. The verdict is the harness's own: a run whose coefficient of variation stays above 0.25 has not settled, and no recovery time can be stated for it – which is a result to report, not a missing number.

metric	value
cockroachdb_dead	settled=True, mean_tps=1,523, fraction_of_baseline=0.4913, cv=0.0587
cockroachdb_recover	settled=False, mean_tps=2,668, fraction_of_baseline=0.7234, cv=0.2702
postgresql_dead	settled=True, mean_tps=947.9, fraction_of_baseline=0.1622, cv=0.1837
postgresql_recover	settled=True, mean_tps=1,032, fraction_of_baseline=0.1654, cv=0.2433

C6. Read and write paths after failover



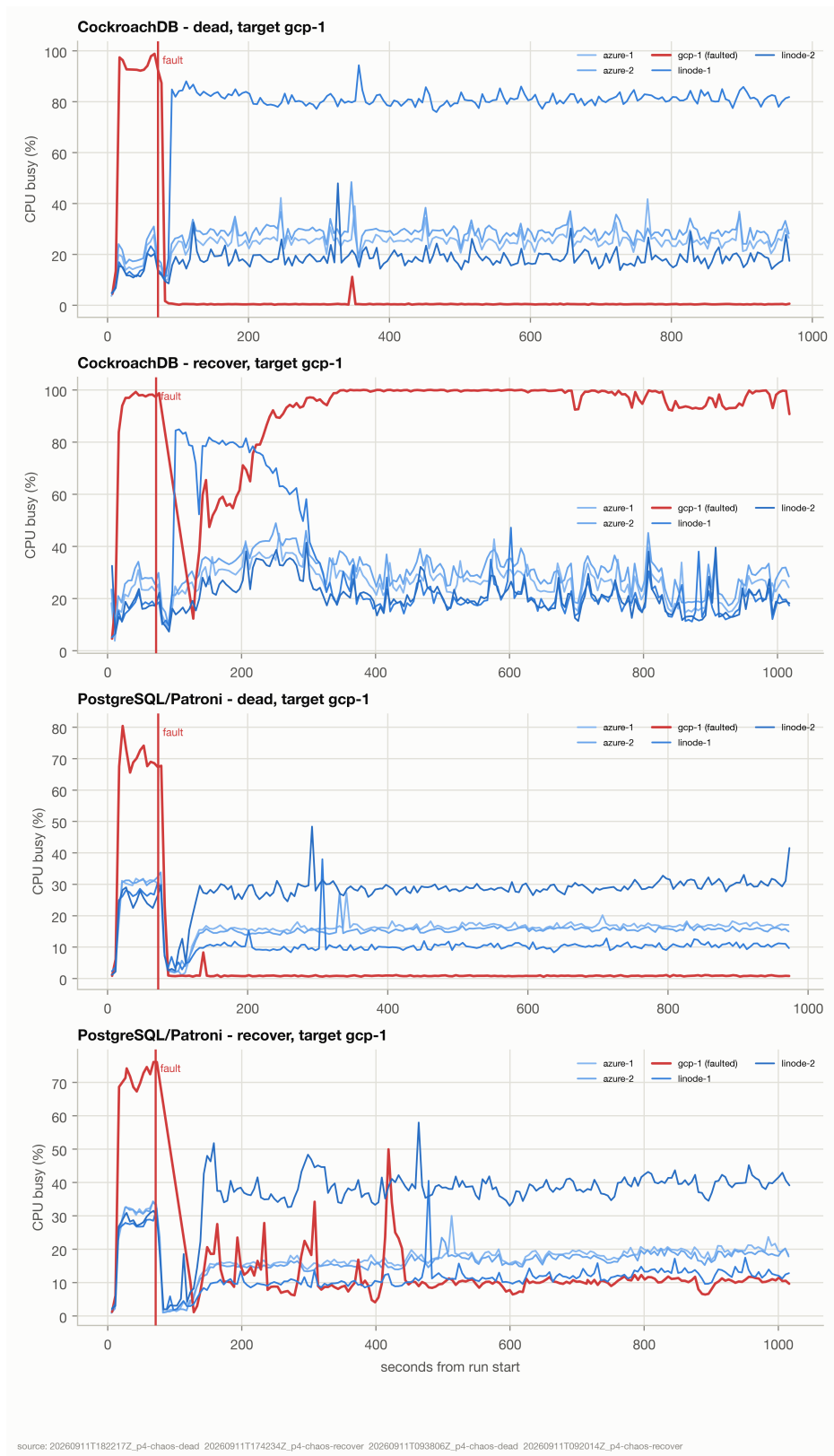
Read and write paths after failover

Read and update medians either side of the fault, log scale. This is the asymmetry the cross-engine write-up must quote carefully: PostgreSQL's clients follow the primary through HAProxy, so a promotion into another region moves the read path too – and reads are 80% of this workload. A throughput RTO that never resolves is then a statement about where the new primary landed, not about the write path, and must be quoted with these read medians beside it.

metric	value
cockroachdb_dead	read_p50_before_ms=13.1, read_p50_after_ms=27.3, update_p50_before_ms=109.1, update_p50_after_ms=218.1
cockroachdb_recover	read_p50_before_ms=10, read_p50_after_ms=19.9,

metric	value
postgresql_dead	update_p50_before_ms=92.3, update_p50_after_ms=100.7
	read_p50_before_ms=1.1, read_p50_after_ms=71.3, update_p50_before_ms=75.5, update_p50_after_ms=234.9
postgresql_recover	read_p50_before_ms=1, read_p50_after_ms=71.3, update_p50_before_ms=75.5, update_p50_after_ms=151

C7. Hardware through the failover

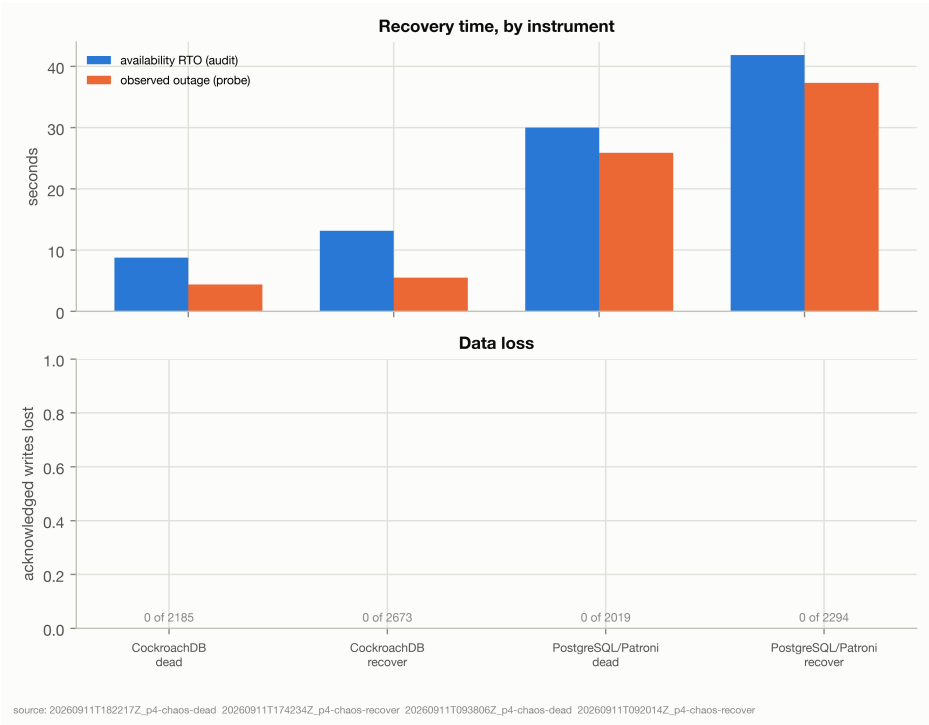


Hardware through the failover

Per-node CPU across the fault, with the faulted node in the reserved status colour. The faulted node going quiet confirms the fault landed on the machine it was aimed at, and a survivor rising afterwards is the promotion visible in hardware rather than inferred from the database’s own logs.

metric	value
cockroachdb_dead_target	gcp-1
cockroachdb_recover_target	gcp-1
postgresql_dead_target	gcp-1
postgresql_recover_target	gcp-1

C8. RTO and RPO summary



RTO and RPO summary

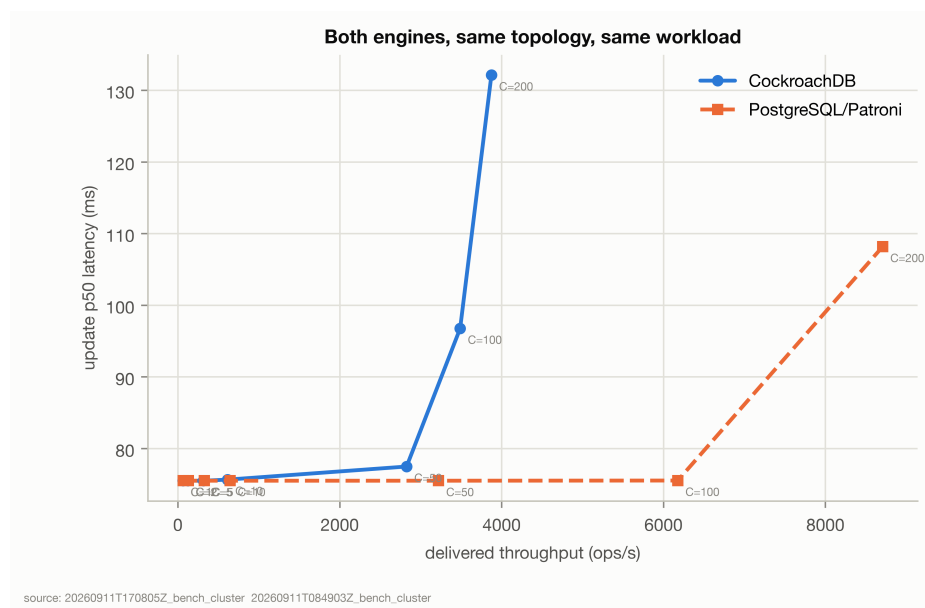
Recovery time from both instruments and acknowledged writes lost, for every engine and fault class. RPO zero is the expected result for a quorum-replicated database and is meaningful only because the measurement could have shown otherwise: the audit client records what it was told committed and advances past ambiguous writes rather than retrying them.

metric	value
CockroachDB_dead	availability_rto_s=8.774, probe_outage_s=4.405, rpo_lost=0, rpo_acknowledged=2185
CockroachDB_recover	availability_rto_s=13.16, probe_outage_s=5.507, rpo_lost=0, rpo_acknowledged=2673
PostgreSQL/Patroni_dead	

metric	value
	availability_rto_s=29.99, probe_outage_s=25.88, rpo_lost=0, rpo_acknowledged=2019
PostgreSQL/Patroni_recover	availability_rto_s=41.86, probe_outage_s=37.32, rpo_lost=0, rpo_acknowledged=2294

D - Engine comparison

D1. Engine throughput-latency curves

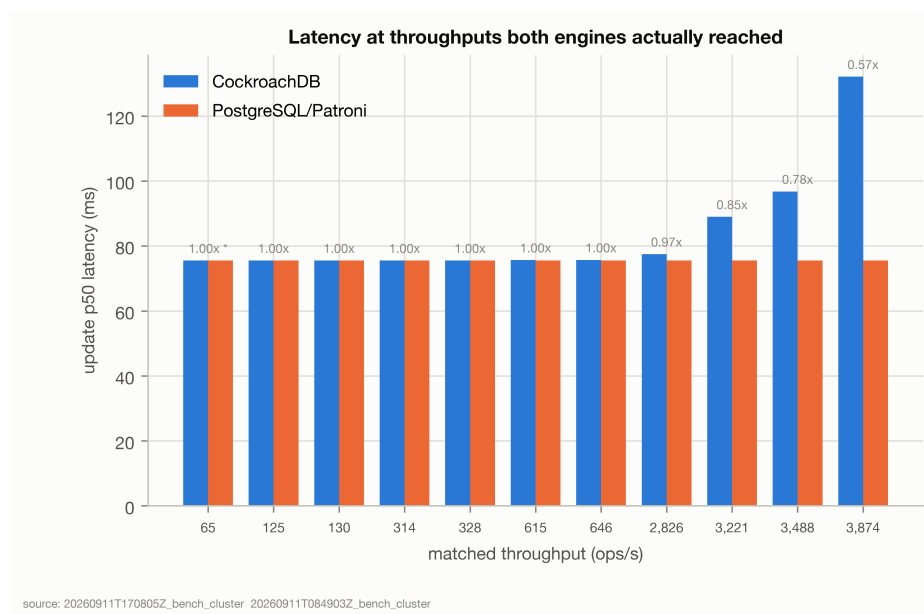


Engine throughput-latency curves

Each engine's own curve, annotated with the concurrency that produced each point. Read the horizontal distance as capacity and the vertical as cost; quote the saturation point rather than a single ratio, because the gap between the curves depends entirely on where along them it is measured.

metric	value
cockroachdb_peak_tps	3,874
cockroachdb_peak_concurrency	200
cockroachdb_saturated	False
postgresql_peak_tps	8,708
postgresql_peak_concurrency	200
postgresql_saturated	False

D2. Latency at matched throughput



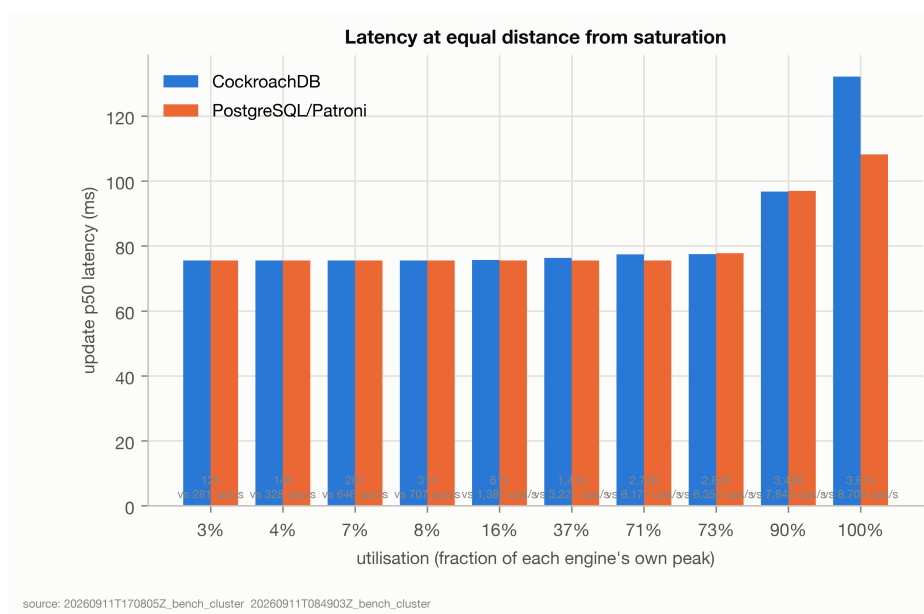
Latency at matched throughput

Both engines evaluated at throughputs each of them genuinely measured – no extrapolation beyond either curve. The ratio above each pair is the overhead at that load; the starred point is the least confounded one, where the two engines were closest to the same fraction of their own capacity. Do not quote the largest ratio: it is the one where the slower engine is nearest saturation and so carries the most of its own queueing.

Field-name note (applies to D2 and D3 below): `phase_ii_*`/`phase_iii_*` are `engine_comparison.py`'s fixed internal labels for the two arms of the comparison – `phase_ii` is always the run passed as `--crdb` (CockroachDB) and `phase_iii` is always the run passed as `--pg` (PostgreSQL) to `crdblab analyze engine-comparison`, regardless of the order the two flags are typed on the command line, since they are named rather than positional arguments. Despite the name, they do not refer to the experiment's Phase II (bench) or Phase III (chaos run --mode recover) – both of D2/D3's numbers are drawn entirely from Phase II (bench) runs of both engines. Confirmed against D1's per-engine peak-tps figures below (`phase_ii_peak_tps 3,874` = `cockroachdb_peak_tps`; `phase_iii_peak_tps 8,708` = `postgresql_peak_tps`).

metric	value
points	11 entries
least_confounded	throughput_tps=65.2, phase_ii_latency_ms=75.5, phase_iii_latency_ms=75.5, overhead_x=1, phase_ii_utilisation=0.017, phase_iii_utilisation=0.007, utilisation_gap=0.009, measured_in=PostgreSQL

D3. Latency at matched utilisation

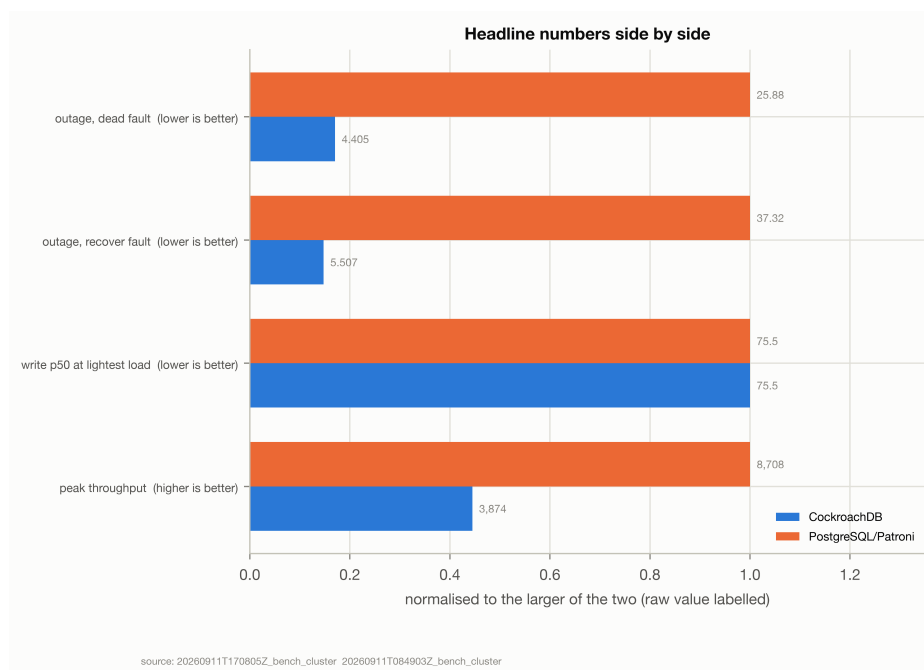


Latency at matched utilisation

The engines held at the same fraction of their own measured capacity, so their queueing components are comparable and the residual is closer to the replication path alone. The throughputs beneath each pair are deliberately different – that is what this framing holds variable – so these bars must never be quoted as a cost at any particular ops/s.

metric	value
points	10 entries
phase_ii_peak_tps (CockroachDB)	3,874
phase_iii_peak_tps (PostgreSQL)	8,708

D4. Engine scorecard

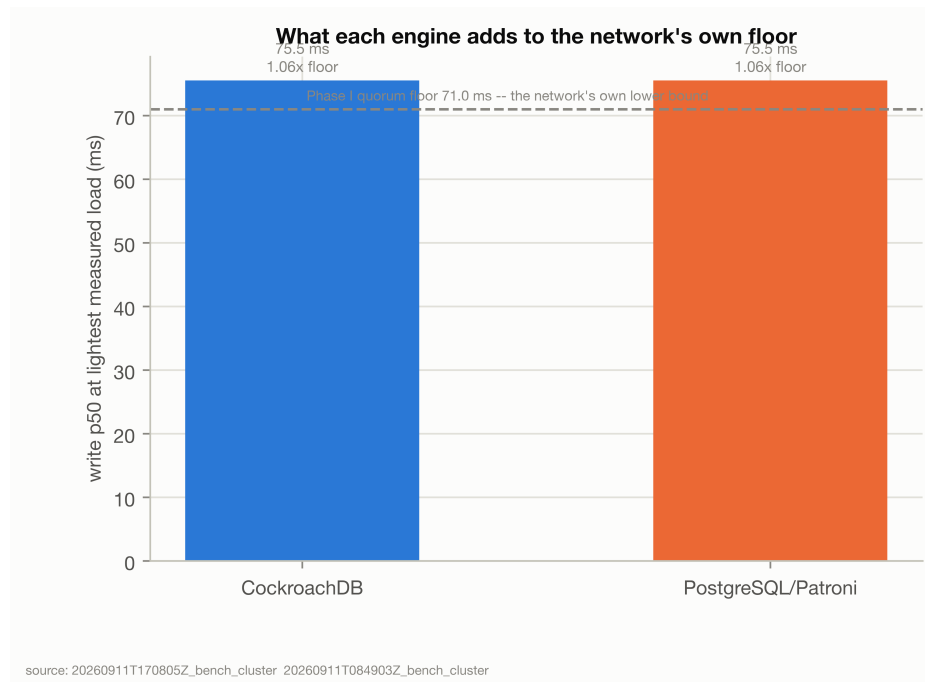


Engine scorecard

Each row normalised to the larger of the two values so that quantities in different units share an axis; the raw value is printed beside every bar because the normalised length alone is not quotable. The direction that counts as better differs by row and is stated on each one.

metric	value
peak throughput	cockroachdb=3,874, postgresql=8,708
write p50 at lightest load	cockroachdb=75.5, postgresql=75.5
outage, recover fault	cockroachdb=5.507, postgresql=37.32
outage, dead fault	cockroachdb=4.405, postgresql=25.88

D5. Cost of consistency



Cost of consistency

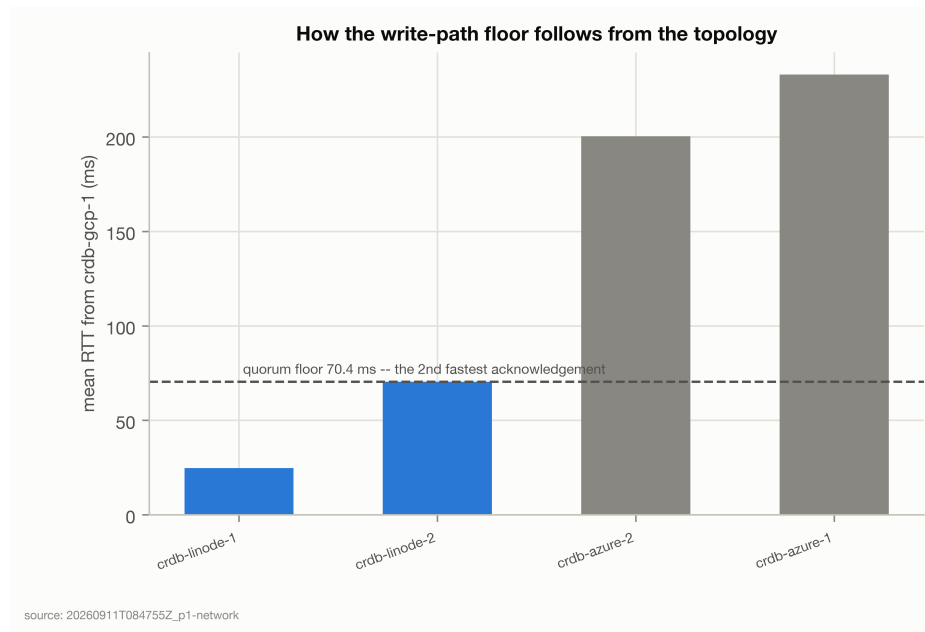
Each engine's write median at its lightest measured load, against the quorum floor measured independently by ping in Phase I. The floor is what the speed of light and this topology cost before any database is involved – a 3-of-5 Raft quorum and Patroni's ANY 2 acknowledgement are the same geometry – so the excess above it is the software's own contribution. Taken at the lightest load because that is where queueing contributes least.

Provenance note: this chart's floor is taken from `ctx.inventory.latest("network", profile=...)` – the single most recent Phase I run at this profile regardless of engine – which for thesis-extended is run 20260911T170557Z_p1-network, taken against the **CockroachDB** deployment. E1 and E4 below instead pick one Phase I run per engine and print the first one encountered, which for this profile is run 20260911T084755Z_p1-network, against the **PostgreSQL** deployment (70.39, not 70.99). Both are genuine, independently-measured Phase I floors for this profile; they differ because each database engine's Terraform deployment redeploys every cluster node, and instructions.md documents that per-deployment RTTs shift by a few ms even when node ordering is stable. Treat the true floor as bracketed at ~70.4-71.0 ms rather than picking one chart's figure over the other's.

metric	value
quorum_floor_ms (CockroachDB-deployment Phase I run 20260911T170557Z_p1-network)	70.99
CockroachDB_write_p50_ms	75.5
CockroachDB_x_over_floor	1.064
PostgreSQL/Patroni_write_p50_ms	75.5
PostgreSQL/Patroni_x_over_floor	1.064

E - Network and provenance

E1. Quorum floor derivation



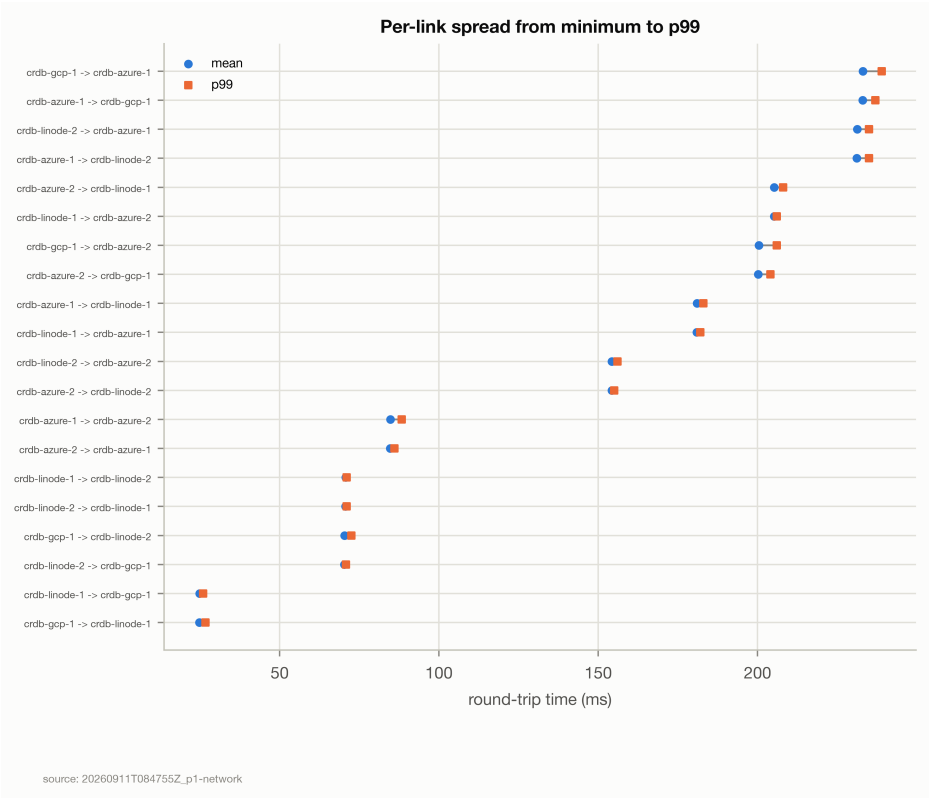
Quorum floor derivation

Round-trip time from the gateway to every other node, sorted. A write commits once the leader and the two fastest acknowledgements have it, so the second bar sets the floor and the slower nodes do not gate an ordinary commit at all. Measured with ping, so it is independent of both databases – which is what lets it bound them.

Provenance note: this figure is drawn from Phase I run 20260911T084755Z_p1-network – the **PostgreSQL**-deployment run, per the filename above and per e1_quorum_floor's per-engine run selection. D5 above instead uses the single most-recently-run Phase I run for this profile regardless of engine, which is the later **CockroachDB**-deployment run 20260911T170557Z_p1-network (70.99, not 70.39). See D5's provenance note for the full explanation; the true floor for this profile is bracketed at ~70.4-71.0 ms across the two deployments.

metric	value
gateway	crdb-gcp-1
recorded_quorum_floor_ms (PostgreSQL-deployment Phase I run 20260911T084755Z_p1-network)	70.39
derived_floor_ms	70.39

E2. Link stability



Link stability

Minimum to p99 for every ordered pair. A short bar is a link whose mean is a real description of it; a long one is a link whose mean is an average over two different behaviours, and any latency figure resting on it inherits that spread. Note that ping's printed precision degrades as RTT grows – each link's own resolution is recorded in network.csv.

metric	value
links	20
max_spread_ms	6.458
worst_link	crdb-gcp-1 -> crdb-azure-1
max_loss_pct	0

E3. Run provenance

Every run considered, and whether it passed its gates

run id	engine	profile	phase	gate
20260911T084755Z_p1-network	postgresql	thesis-extended	network	loads and validates
20260911T084903Z_bench_cluster	postgresql	thesis-extended	bench	loads and validates
20260911T092014Z_p4-chaos-recover	postgresql	thesis-extended	chaos-recover	loads and validates
20260911T093806Z_p4-chaos-dead	postgresql	thesis-extended	chaos-dead	loads and validates
20260911T170557Z_p1-network	cockroachdb	thesis-extended	network	loads and validates
20260911T170805Z_bench_cluster	cockroachdb	thesis-extended	bench	loads and validates
20260911T174234Z_p4-chaos-recover	cockroachdb	thesis-extended	chaos-recover	loads and validates
20260911T182217Z_p4-chaos-dead	cockroachdb	thesis-extended	chaos-dead	loads and validates

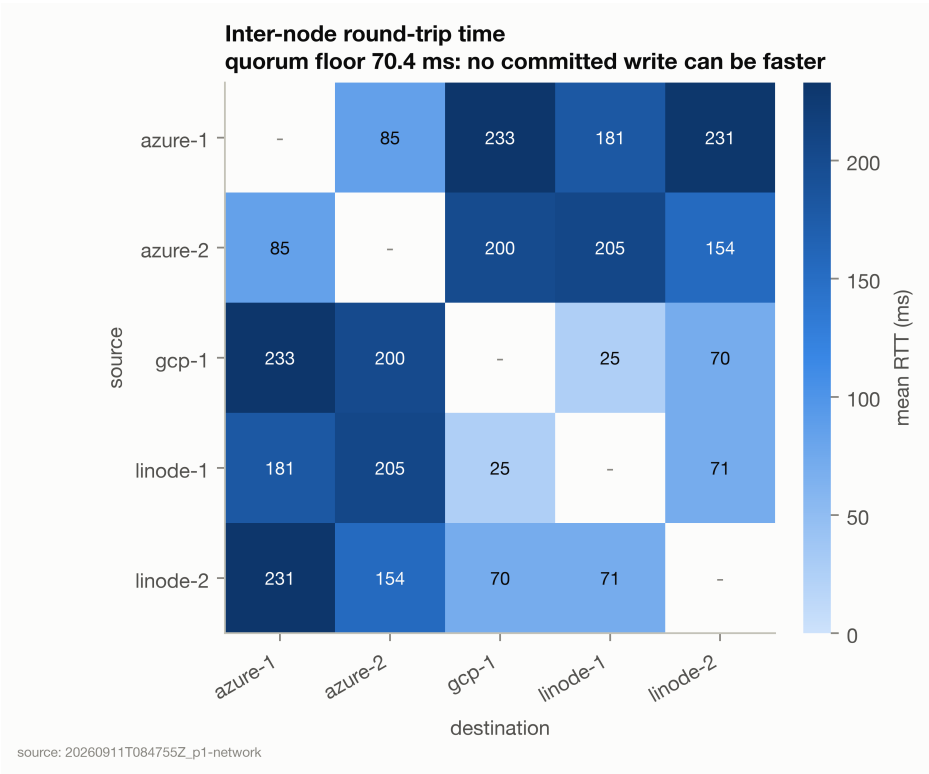
source: no run

Run provenance

Every run directory this report considered. A run reaches a chart only by loading through the gated loader, which refuses anything without a manifest, with unexpected columns, or that fails pre-flight or validation – so a run listed as refused here contributed to nothing in this document.

metric	value
runs_considered	8
runs_passing	8

E4. Round-trip matrix



Round-trip matrix

Mean round-trip time between every ordered pair of nodes, including the two neither E1 nor E2 puts on one axis with the rest: the non-gateway pairs. Darker is slower; a node does not ping itself, shown as a dash.

Provenance note: same run as E1 – Phase I run 20260911T084755Z_p1-network, the **PostgreSQL**-deployment Phase I run for this profile. See D5’s provenance note: D5 draws its floor from the later **CockroachDB**-deployment run instead (70.99), and the two together bracket this profile’s floor at ~70.4-71.0 ms.

metric	value
quorum_floor_ms (PostgreSQL-deployment Phase I run 20260911T084755Z_p1-network)	70.39